MINI PROJECT REPORT

Think-or-Stop

Design and Evaluation of an Adaptive Early-Exit CNN for Efficient Image Classification

A convolutional neural network with three classifiers at different depths that can stop early for easy images and keep thinking for hard ones, trained from scratch in PyTorch and evaluated for accuracy, computation, GPU latency, calibration and robustness on MNIST and CIFAR-10.

Student name	
Roll number / ID	
Course / Subject	Deep Learning
Guide / Supervisor	
Institution	
Date	October 2026

Hardware: NVIDIA GeForce RTX 5060 Laptop GPU (7.93 GB) · PyTorch 2.14.1+cu130 · CUDA 13.0 · Python 3.14.7 · Windows 11

Contents

Abstract 4

1. Introduction 5

1.1 Problem statement . . . 5
1.2 Idea: think, or stop . . . 5
1.3 Research questions . . . 5
1.4 Scope and constraints . . . 5
1.5 Related work . . . 6

2. Background concepts 7

2.1 Convolution . . . 7
2.2 Batch normalisation (BN) . . . 7
2.3 ReLU activation . . . 7
2.4 Max pooling . . . 7
2.5 Fully connected layers, dropout and softmax . . . 7
2.6 Cross-entropy loss and backpropagation . . . 7
2.7 Optimisers: SGD with momentum and Adam . . . 8
2.8 Early-exit networks . . . 8
2.9 Calibration . . . 8
2.10 MACs, FLOPs and latency . . . 8

3. Datasets and preprocessing 9

3.1 MNIST (development and smoke testing) . . . 9
3.2 CIFAR-10 (main benchmark) . . . 9
3.3 Train / validation / test splits . . . 9
3.4 Normalisation . . . 9
3.5 Data augmentation (CIFAR-10 training only) . . . 9
3.6 Corrupted test images . . . 10

4. Model architecture 11

4.1 Overview . . . 11
4.2 Layer-by-layer details (CIFAR-10) . . . 12
4.3 Two model variants with a fair comparison . . . 12
4.4 Weight initialisation . . . 12

5. Training method 13

5.1 Multi-exit loss . . . 13
5.2 Hyperparameters . . . 13
5.3 Checkpointing, resuming and reproducibility . . . 13
5.4 GPU usage . . . 14
5.5 Training curves . . . 14

6. Inference: fixed exits, adaptive exits and threshold selection 15

6.1 Adaptive (confidence-based) inference . . . 15
6.2 Batched implementation . . . 15

6.3 Verifying correctness . . . 15
6.4 Cost accounting . . . 15
6.5 Selecting the threshold on validation data . . . 16
7. Evaluation methodology 17
7.1 Metrics . . . 17
7.2 Latency measurement protocol . . . 17
7.3 Multiple seeds . . . 18
8. Results 19
8.1 Main comparison on CIFAR-10 . . . 19
8.2 Results per seed . . . 19
8.3 Effect of the threshold (RQ2, RQ3) . . . 20
8.4 Which images stop early? . . . 22
8.5 Per-class accuracy . . . 22
8.6 Calibration (RQ6) . . . 22
8.7 Latency on the GPU (RQ4) . . . 23
8.8 Robustness to corruption (RQ5) . . . 24
8.9 MNIST smoke test . . . 25
9. Answers to the research questions 26
9.1 Overall conclusion . . . 26
10. Limitations and future work 27
10.1 Limitations . . . 27
10.2 Future work . . . 27
11. Implementation 28
11.1 Repository structure . . . 28
11.2 Testing . . . 28
11.3 Main commands . . . 29
12. Viva and cross-question preparation 30
12.1 Concept questions . . . 30
12.2 Methodology questions . . . 31
12.3 Results questions . . . 32
12.4 Implementation questions . . . 33
References 34
Appendix A. Glossary 35
Appendix B. Environment 35
Appendix C. Confusion matrix 36

Abstract

A conventional convolutional neural network (CNN) spends exactly the same amount of computation on every image, whether the image is an obvious, clean example or a confusing one. **Think-or-Stop** studies a simple alternative: attach extra classifiers (*exits*) after the early and middle parts of the network, and let each image stop at the first exit that is confident enough. Easy images should finish early; hard images continue to the full network.

We implemented a compact three-stage CNN from scratch in PyTorch (no pretrained weights), trained a conventional single-exit baseline and a three-exit model on MNIST (smoke test) and CIFAR-10 (main benchmark), and evaluated them with three random seeds each. The confidence threshold for early exiting was selected on a held-out validation split only and then applied unchanged to the test set.

On CIFAR-10 the standard CNN reached **90.37 ± 0.10%** test accuracy. The multi-exit model's final exit reached 90.11 ± 0.20%. With the validation-selected threshold ($\tau \approx 0.85$), adaptive inference reached **89.28 ± 0.24%** while using only **63.5 ± 0.8%** of the full network's estimated multiply-accumulate operations (MACs). At the same computation budget, always stopping at Exit 2 gives only 85.89 ± 0.33%, so choosing the depth per image is clearly better than a fixed shallower network.

However, the saved arithmetic did **not** become saved time on the GPU: adaptive inference was slower than simply running the full network (0.96 ± 0.03 vs 0.61 ± 0.01 ms per image at batch size 1), because the confidence checks and batch splitting add overhead that a GPU does not hide. Adaptive inference was also better calibrated than the final exit (ECE 0.027 ± 0.003 vs 0.041 ± 0.002), and under synthetic noise and blur it automatically sent more images deeper while keeping accuracy close to full depth. These results show both the promise and the practical limits of early exiting.

> **Note on novelty**
> Early exiting is an **established research technique** (BranchyNet 2016, Multi-Scale DenseNet 2018, Shallow-Deep Networks 2019). This project does not claim a new method. Its contribution is a clean, tested implementation and a careful, honest evaluation, including the cases where the adaptive model does worse.

1. Introduction

1.1 Problem statement

Deep networks are accurate but expensive. Every image goes through every layer, even though many images are easy: a clear photo of a ship on blue water is recognisable from simple colour and edge features, while a blurry cat that looks like a dog needs the deeper, more abstract features. Running the full network on every image wastes computation on the easy cases. This matters on phones, embedded devices and servers that classify millions of images.

1.2 Idea: think, or stop

We split the network into three stages and place a small classifier after each one. During inference, after each early stage we look at the classifier's output probabilities. If the most likely class has a probability of at least a threshold τ (for example 0.9), the network *stops* and outputs that class. Otherwise it *thinks more*: it runs the next stage. The last classifier always answers. The threshold controls the trade-off: a low τ stops early often (fast, less accurate); a high τ rarely stops early (slower, more accurate).

1.3 Research questions

The project was designed to answer six questions experimentally, without assuming the adaptive model wins:

- **RQ1.** How does a conventional CNN compare with a CNN containing intermediate classifiers?
- **RQ2.** How does the exit-confidence threshold affect accuracy and computational cost?
- **RQ3.** Can early exits reduce the average amount of computation?
- **RQ4.** Does early-exit inference actually reduce measured latency on the GPU?
- **RQ5.** Do early exits remain reliable when images are noisy or corrupted?
- **RQ6.** How do accuracy, calibration, latency and exit rate change under different operating conditions?

1.4 Scope and constraints

- Framework: PyTorch, Python, trained from scratch on an NVIDIA RTX 5060 Laptop GPU (8 GB) under Windows 11.
- No pretrained models or weights, no paid or external AI APIs, no Docker or WSL.
- Datasets downloaded only through official mechanisms (torchvision for MNIST and CIFAR-10).
- MNIST is used for fast development and debugging; CIFAR-10 is the main benchmark.
- The test set is never used for training, checkpoint selection or threshold selection.

1.5 Related work

BranchyNet (Teerapittayanon, McDanel and Kung, 2016) added side-branch classifiers to standard networks and exited when the entropy of the prediction was low. **Multi-Scale DenseNet** (Huang et al., 2018) designed an architecture specifically for anytime and budgeted prediction with many exits. **Shallow-Deep Networks** (Kaya, Hong and Dumitras, 2019) showed that internal classifiers can reduce 'overthinking', where deeper layers change a correct early answer into a wrong one. **Guo et al.** (2017) showed that modern deep networks are often overconfident and introduced the widely used Expected Calibration Error measurement. **Hendrycks and Dietterich** (2019) introduced CIFAR-10-C to benchmark robustness to common corruptions. Think-or-Stop uses the simplest form of these ideas: maximum softmax probability as the confidence score and one shared threshold.

2. Background concepts

This chapter explains every building block used in the project, so that each design decision can be justified.

2.1 Convolution

A convolutional layer slides small learnable filters (here 3×3) over the image. Each filter computes a weighted sum of a small neighbourhood at every position, producing a *feature map* that responds to a pattern (an edge, a colour blob, later a texture or an object part). Convolution uses far fewer parameters than a fully connected layer because the same filter is reused at every position (weight sharing), and it keeps spatial structure. We use padding 1 so a 3×3 convolution keeps the height and width unchanged.

2.2 Batch normalisation (BN)

BN normalises each channel's activations using the mean and variance of the current mini-batch, then applies a learnable scale and shift. It stabilises and speeds up training and allows larger learning rates. In evaluation mode BN uses *running averages* collected during training instead of batch statistics, so a sample's output does not depend on the other samples in its batch. This property is essential for our batched early-exit implementation (Section 6.2).

2.3 ReLU activation

ReLU(x) = max(0, x). It adds non-linearity so the network can learn complex functions, is cheap to compute, and does not saturate for positive values, which reduces vanishing gradients compared with sigmoid or tanh.

2.4 Max pooling

2×2 max pooling keeps the largest value in each 2×2 window, halving height and width. It reduces computation in later layers, gives some tolerance to small shifts, and increases the region of the image each later neuron 'sees' (its receptive field).

2.5 Fully connected layers, dropout and softmax

A fully connected (linear) layer connects every input to every output; we use two of them in each classifier head. **Dropout** randomly sets a fraction p (here 0.3) of activations to zero during training so the network cannot rely on any single feature; this reduces overfitting. Dropout is switched off at evaluation time. The final layer outputs 10 *logits* $z_1 \ldots z_{10}$. **Softmax** converts them to probabilities that sum to one:

$$p_k = \exp(z_k) \,/\, \Sigma_j \exp(z_j)$$

2.6 Cross-entropy loss and backpropagation

For an image with true class y, the cross-entropy loss is $L = -\log p_y$: small when the network gives the correct class high probability, large when it does not. **Forward propagation** computes the prediction and loss; **backpropagation** applies the chain rule backwards through every layer to compute the gradient of the loss with respect to every weight; the **optimiser** then updates the weights in the direction that reduces the loss.

2.7 Optimisers: SGD with momentum and Adam

SGD with momentum moves weights against the gradient and keeps a running 'velocity' so updates continue smoothly in consistent directions. **Adam** additionally keeps a running estimate of each weight's squared gradient and scales each weight's step individually, which makes it less sensitive to the learning rate and usually fast to converge. Both are implemented; Adam (learning rate 0.001) is the default used for all reported results.

2.8 Early-exit networks

An early-exit network has classifiers at several depths that share one backbone. It is trained so every exit is useful, and at inference a *policy* decides where each input stops. Our policy uses the **maximum softmax probability** (MSP) as a confidence score. MSP is only a *proxy* for correctness: a network can be confidently wrong, especially on unusual or corrupted inputs.

2.9 Calibration

A classifier is *calibrated* if, among all predictions made with 80% confidence, about 80% are correct. Calibration matters here because the exit decision trusts the confidence value. If an early exit is overconfident, it will stop on images it gets wrong. We measure calibration with Expected Calibration Error (Section 7.1).

2.10 MACs, FLOPs and latency

A **multiply-accumulate** (MAC) is one multiplication followed by one addition, the core operation of convolutions and linear layers; FLOPs ≈ 2 × MACs. MACs are a hardware-independent estimate of arithmetic work. **Latency** is the measured wall-clock time. They are not the same thing: a GPU runs thousands of operations in parallel, so latency is often limited by memory traffic, the number of separate kernel launches and CPU–GPU synchronisation rather than by arithmetic. Fewer MACs therefore do not guarantee lower latency, which is why this project measures both.

3. Datasets and preprocessing

3.1 MNIST (development and smoke testing)

70,000 grayscale 28×28 images of handwritten digits 0–9: 60,000 official training images and 10,000 official test images. It trains in seconds and was used to debug the full pipeline before the main experiments.

3.2 CIFAR-10 (main benchmark)

60,000 colour 32×32 images in 10 classes (airplane, automobile, bird, cat, deer, dog, frog, horse, ship, truck): 50,000 official training and 10,000 official test images, 6,000 per class. Images are tiny and objects vary in pose, scale and background, so CIFAR-10 is much harder than MNIST and shows clear differences between exits.

3.3 Train / validation / test splits

Dataset	Training	Validation	Test	Image	Classes
MNIST	55,000	5,000	10,000 (official)	1×28×28	10
CIFAR-10	45,000	5,000	10,000 (official)	3×32×32	10

Table 1. Data splits used in every experiment.

The validation set is held out from the official training set using a random permutation with a fixed **split seed (42)**. The split seed is deliberately separate from the training seed, so all three training seeds see exactly the same validation images and the comparison between seeds is fair. A unit test confirms the split is reproducible, disjoint and complete.

> **Why a separate validation set?**
> The validation set is used for two decisions: choosing the best epoch checkpoint and choosing the confidence threshold. The test set is touched only once, for final reporting. If we chose the threshold by looking at test accuracy and then reported that same test accuracy, the result would be optimistically biased because the test data would have influenced the model's settings.

3.4 Normalisation

Each channel is standardised as (x − mean) / std using the published per-channel statistics of the official training set: MNIST mean 0.1307, std 0.3081; CIFAR-10 mean (0.4914, 0.4822, 0.4465), std (0.2470, 0.2435, 0.2616). Centring inputs around zero with unit variance helps optimisation. (Honest detail: these constants include the 5,000 validation images, a negligible effect that does not involve test data.)

3.5 Data augmentation (CIFAR-10 training only)

- **Random crop**: pad the image by 4 pixels on each side, then crop a random 32×32 window, so the network learns that objects can appear at slightly different positions.
- **Random horizontal flip** with probability 0.5: a left-facing horse is still a horse.
- Validation and test images are never augmented, so evaluation is deterministic.
- MNIST is not augmented: horizontal flips would change digit identity and the task is already easy.

3.6 Corrupted test images

To test reliability under distribution shift, we created **synthetic corruptions** of the test set, re-implemented with the severity parameters published for CIFAR-10-C:

Corruption	What it does	Severity 1	Severity 3	Severity 5
Gaussian noise	adds $N(0, \sigma^2)$ to pixel values in [0,1], then clips	$\sigma = 0.04$	$\sigma = 0.08$	$\sigma = 0.10$
Gaussian blur	blurs with a Gaussian kernel of std σ pixels	$\sigma = 0.4$	$\sigma = 0.7$	$\sigma = 1.0$

Table 2. Synthetic corruptions used for the robustness experiment.

Each image's noise is generated from its own seeded random generator, so corrupted images are identical across runs, models and batch sizes. Thresholds are still chosen on *clean* validation data, and no corrupted labels are used to tune anything. These synthetic results are reported separately from the real **CIFAR-10-C** benchmark (Hendrycks and Dietterich 2019; Zenodo record 2535967, licence CC BY 4.0, 2.9 GB, MD5 56bf5dcef84df0e2308c6dcbcbbd8499). Support for CIFAR-10-C is implemented, with checksum verification, but it was not downloaded or evaluated in this project because it is an optional large download.

4. Model architecture

4.1 Overview

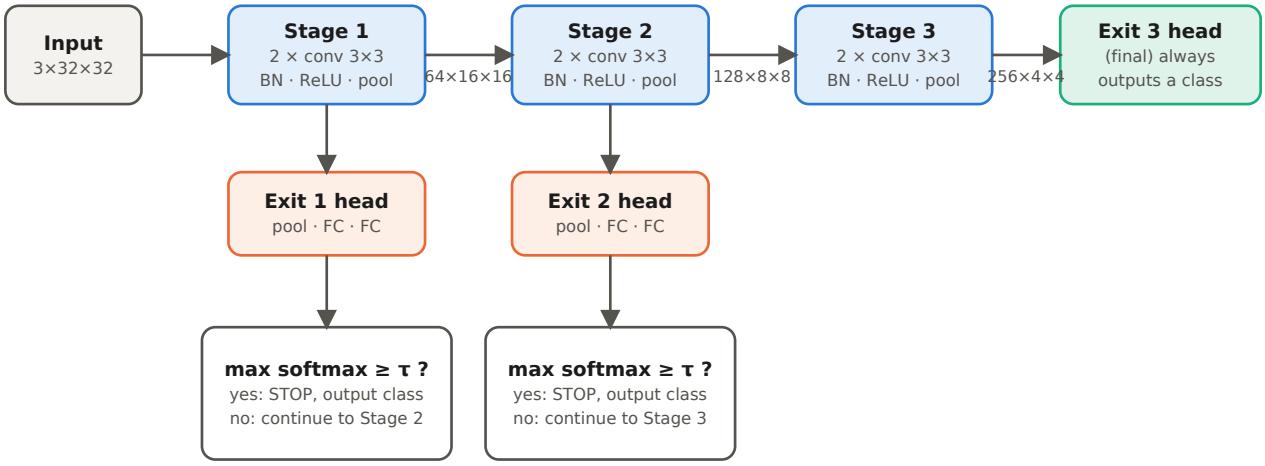


Figure 1. Think-or-Stop architecture (CIFAR-10 shapes). After Stages 1 and 2 an exit head predicts; if its confidence reaches τ the image stops, otherwise it continues to the next stage. Exit 3 always answers.

The network has a shared backbone of three stages. Each stage is two Conv(3×3)–BatchNorm–ReLU blocks followed by 2×2 max pooling. The width doubles at each stage (64 → 128 → 256 channels) while the spatial size halves (32 → 16 → 8 → 4), a standard VGG-style pattern: early layers detect simple local patterns at high resolution, later layers combine them into more abstract features at lower resolution.

Each exit head is: AdaptiveAvgPool(2×2) → Flatten → Linear(4c → 128) → ReLU → Dropout(0.3) → Linear(128 → 10), where c is the number of channels at that stage. Adaptive pooling makes the head independent of the input size, so the same code works for 28×28 MNIST (feature maps 14 → 7 → 3) and 32×32 CIFAR-10 (16 → 8 → 4) and keeps each head small.

4.2 Layer-by-layer details (CIFAR-10)

Block	Layers	Output shape	Parameters	MACs per image
Input	–	3×32×32	–	–
Stage 1	Conv 3→64, BN, ReLU; Conv 64→64, BN, ReLU; MaxPool	64×16×16	38,848	39,518,208
Exit 1 head	AvgPool 2×2; FC 256→128; ReLU; Dropout; FC 128→10	10 logits	34,186	34,048
Stage 2	Conv 64→128, BN, ReLU; Conv 128→128, BN, ReLU; MaxPool	128×8×8	221,696	56,623,104
Exit 2 head	AvgPool 2×2; FC 512→128; ReLU; Dropout; FC 128→10	10 logits	66,954	66,816
Stage 3	Conv 128→256, BN, ReLU; Conv 256→256, BN, ReLU; MaxPool	256×4×4	885,760	56,623,104
Exit 3 head	AvgPool 2×2; FC 1024→128; ReLU; Dropout; FC 128→10	10 logits	132,490	132,352
Total (multi-exit)			1,379,934	152,997,632 (all heads)
Total (baseline)	same stages + Exit 3 head only		1,278,794	152,896,768

Table 3. Architecture, parameter counts and MACs (computed by the code).

Worked example of the MAC count for the first convolution: output 64 channels × 32 × 32 positions, each needing 3 input channels × 3 × 3 kernel = 27 multiply-adds: 64 × 1024 × 27 = 1,769,472 MACs. The second convolution of Stage 1 needs 64 × 1024 × (64 × 9) = 37,748,736 MACs, so Stage 1 totals 39,518,208. Because Stage 1 works at full resolution, it already costs about 26% of the whole network even though it has only 3% of the parameters.

4.3 Two model variants with a fair comparison

- **Model A, standard CNN (baseline)**: the same three stages and the same final head, but no Exit 1 or Exit 2. It is trained with ordinary cross-entropy on the final output.
- **Model B, multi-exit CNN**: identical backbone plus the two intermediate heads, trained with a weighted sum of the three losses.
- **Model C, fixed-exit inference**: Model B evaluated by always using Exit 1, Exit 2 or Exit 3, running only the stages that exit needs.
- **Model D, adaptive inference**: Model B with the confidence-threshold policy, either with a manually chosen $\tau = 0.9$ or with τ selected on validation data.

Because the baseline and the multi-exit model share exactly the same backbone and final head, any difference between them is caused only by the extra heads and the training objective. The extra heads add 101,140 parameters (7.9%) and only 100,864 MACs (0.07%).

4.4 Weight initialisation

Convolution weights use Kaiming (He) normal initialisation, which keeps activation variance stable through ReLU layers. BN starts as the identity (scale 1, shift 0). Linear layers keep PyTorch's default initialisation, which gives initial logits near zero so the initial loss is about $\ln 10 \approx 2.30$ per exit. This was verified during development: an earlier ReLU-gain initialisation of the linear layers produced initial losses of about 4 per exit and was corrected.

5. Training method

5.1 Multi-exit loss

All three exits are trained together. For a mini-batch, each exit's cross-entropy is computed and the total loss is the weighted sum:

$$L_{total} = w_1 \cdot L_{exit1} + w_2 \cdot L_{exit2} + w_3 \cdot L_{exit3}$$

We use equal weights w = (1, 1, 1), the most neutral choice: every exit receives an equal share of the gradient. Stage 1 therefore receives gradients from all three heads, Stage 2 from two, and Stage 3 only from the final head. A unit test checks that every head and every backbone parameter receives a non-zero gradient. The weights are configurable with --exit-weights; a zero weight is rejected for the multi-exit model because that head would never learn.

5.2 Hyperparameters

Setting	Value	Reason
Optimiser	Adam	robust default, fast convergence; SGD+momentum also supported
Learning rate	0.001	standard Adam starting value
LR schedule	cosine annealing to 0 over all epochs	smooth decay, no extra tuning
Weight decay	1e-4	mild L2 regularisation
Batch size	128	fits easily in 8 GB (peak about 0.9 GB)
Epochs	30 (CIFAR-10), 2 (MNIST smoke run)	enough for the curves to flatten (Figure 2)
Dropout in heads	0.3	reduces overfitting in the fully connected layers
Exit loss weights	(1, 1, 1)	neutral; configurable
Seeds	0, 1, 2 (training); 42 (split)	mean ± std over independent runs
Checkpoint rule	highest final-exit validation accuracy	same rule for both models

Table 4. Training configuration used for all reported CIFAR-10 runs.

5.3 Checkpointing, resuming and reproducibility

- After every epoch, last.pt is saved (model, optimiser, scheduler, epoch, history, random-number state); best.pt is saved whenever the final-exit validation accuracy improves. Both models use the same rule, so neither gets an advantage from checkpoint selection.
- Training can be resumed exactly from last.pt with --resume. A test trains, interrupts and resumes a run and checks the epoch count and learning rate.
- Python, NumPy and PyTorch (CPU and GPU) are seeded. cuDNN autotuning is on for speed, so GPU runs with the same seed are statistically, not bit-for-bit, identical; --deterministic forces exact reproducibility at some speed cost.
- All settings live in one configuration class and are exposed as command-line flags; nothing is hard-coded.

5.4 GPU usage

Training ran on CUDA. The code checks that CUDA is really available (and stops with a clear error instead of silently falling back to CPU), prints the GPU name and versions, and logs the device of the model parameters and of the first batch (both cuda:0 in every run). Data loading uses pinned memory and non-blocking transfers. Peak GPU memory was about 0.9 GB in the first epoch and about 0.4 GB afterwards, far below the 8 GB limit. Mixed precision (--amp) is implemented but was not used for the reported results, so all numbers are full float32.

After the first epoch (which includes start-up costs), a CIFAR-10 epoch took about 5.8 s for the multi-exit model and 5.4 s for the baseline (medians; the baseline's figure is inflated by one seed that ran while the laptop was busy, so these training times should not be compared as a result).

5.5 Training curves

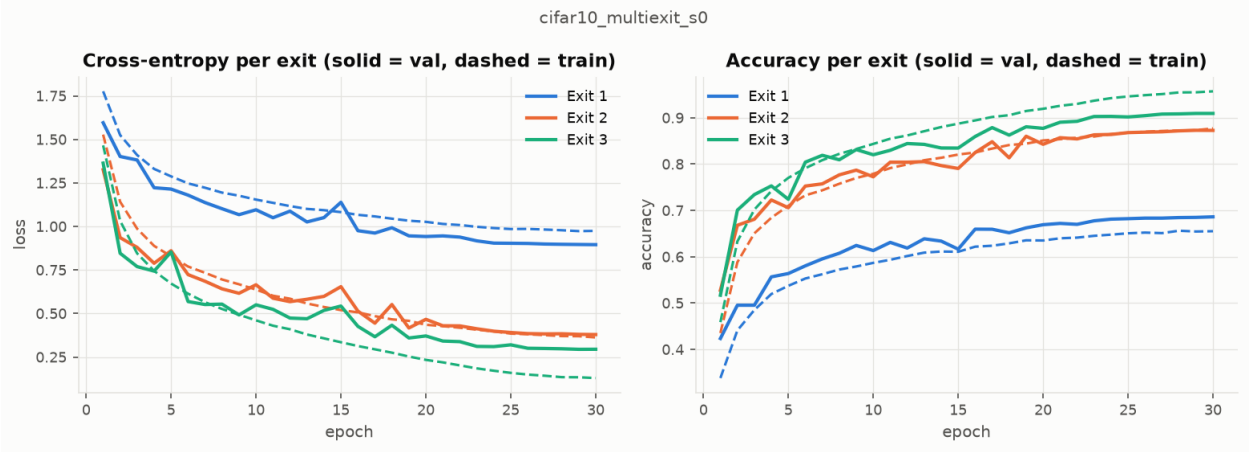


Figure 2. Multi-exit model, seed 0: loss and accuracy per exit (solid = validation, dashed = training).

All three exits improve steadily and the curves flatten by epoch 30. Exit 1 plateaus near 69% because two convolutions do not have enough capacity or receptive field for CIFAR-10. Exit 3 training accuracy (≈96%) is above validation accuracy (≈91%), a moderate generalisation gap; validation accuracy still rose until the end, so the model is not overfitting badly. Interestingly, Exit 1 has *higher* validation than training accuracy, because training accuracy is measured with augmentation and dropout switched on.

6. Inference: fixed exits, adaptive exits and threshold selection

6.1 Adaptive (confidence-based) inference

For each image: run Stage 1 and Exit 1, compute softmax probabilities and their maximum (the confidence). If confidence ≥ τ_1, output Exit 1's class and stop. Otherwise run Stage 2 and Exit 2 and apply the same test with τ_2. If still not confident, run Stage 3 and output Exit 3's class. In this project $\tau_1 = \tau_2 = \tau$ (one shared threshold), but the code also accepts one threshold per exit.

6.2 Batched implementation

```
active = [0, 1, ..., N-1]          # original positions of samples still running
h = x
for each stage s:
    h = stage_s(h)                   # only the still-active samples are processed
    probs = softmax(head_s(h)); conf, pred = max(probs)
    if s is the last stage: done = all samples
    else:                   done = conf >= tau_s
    write pred/conf/exit=s into the output slots of active[done]
    keep only the samples that are not done: active = active[~done]; h = h[~done]
    stop the loop if no samples remain
```

Processing a batch is harder than one image because different images stop at different exits. After each exit, samples that are done are removed from the batch, so deeper stages run only on the remaining ones. The array **active** remembers each remaining sample's original position so results are written back in the correct order. Two optimisations keep the overhead small: the code synchronises with the GPU only once per exit (to count how many samples finished), and it skips the indexing work entirely when no sample, or every sample, stops at that exit. Inference runs in torch.inference_mode(), so no gradients or computation graphs are kept, and the model must be in evaluation mode (checked by the code).

6.3 Verifying correctness

Unit tests compare the batched result with a simple one-image-at-a-time reference, shuffle the batch to confirm that each result returns to the right position, attach hooks to each stage to prove that exited samples are not processed by deeper stages, and check the incomplete final batch, single images, determinism and the absence of gradients. On the real CIFAR-10 test set, the batched implementation and a simulation from stored outputs gave **100% identical predictions and exit decisions** for every seed and threshold. With $\tau = 0.84$ (seed 0), Stage 1 processed 10,000 images, Stage 2 only 7,317 and Stage 3 only 2,597.

6.4 Cost accounting

Fixed exit k costs the stages up to k plus head k. An image that leaves adaptively at exit k costs the stages up to k plus *every head evaluated on the way*, because the earlier heads were computed even though they did not stop the image. The average adaptive cost is the mean of these per-image costs. This is why the adaptive cost at $\tau = 1.0$ (nothing exits early) is 100.07% of the full network, slightly more than running the full network directly.

6.5 Selecting the threshold on validation data

- Evaluate a grid of thresholds (0.00, 0.01, …, 1.00 plus 0.991–0.9999) on the **validation** outputs.
- Constraint (relative mode, used here): validation accuracy ≥ full-depth validation accuracy − 1 percentage point. An absolute mode (accuracy ≥ a fixed target) is also implemented.
- Among thresholds meeting the constraint, choose the one with the **lowest average MACs**; ties go to higher accuracy, then to the higher (safer) threshold.
- If no threshold meets the constraint, fall back to 'never exit early' and report the constraint as not satisfied. (In relative mode full depth always meets it, so the fallback can only occur in absolute mode.)
- Save the chosen threshold and the full validation sweep, then apply the threshold unchanged to the test set and to the corrupted test sets.

Seed	Selected τ	Full-depth val acc	Required val acc	Adaptive val acc	Val MACs (% full)	Constraint met
0	0.84	90.94%	89.94%	89.98%	61.9%	yes
1	0.86	90.88%	89.88%	89.94%	63.5%	yes
2	0.85	91.42%	90.42%	90.50%	63.4%	yes

Table 5. Threshold selection on the validation split (never on test).

The three independently trained models selected very similar thresholds (0.84, 0.86, 0.85), which suggests the selection procedure is stable.

7. Evaluation methodology

7.1 Metrics

- **Accuracy** for every exit, for the adaptive policy and for the baseline; **per-class accuracy**, **confusion matrix** and a **classification report** (precision, recall, F1 per class).
- **Exit distribution**: percentage of images leaving at Exit 1, 2 and 3; **mean exit depth** (1 = everything stops at Exit 1, 3 = everything goes to the end).
- **Estimated computation**: average MACs per image, also shown as a percentage of the full network.
- **Early-exit error rate**: the error rate among only the images that stopped at Exit 1 or 2.
- **Latency and throughput** on the GPU, and peak GPU memory.
- **Calibration**: Expected Calibration Error and reliability diagrams; also negative log-likelihood.

Expected Calibration Error (ECE). Predictions are grouped into 15 equal-width confidence bins (0–1/15, ..., 14/15–1). In each bin B we compare the average confidence with the actual accuracy:

$$\mathrm{ECE} = \Sigma_b \, (|B_b| / N) \cdot | \, \mathrm{acc}(B_b) - \mathrm{conf}(B_b) \, |$$

ECE = 0 means perfectly calibrated; larger values mean the confidence is misleading. A unit test checks the implementation on a hand-computed example (four predictions at 0.9 confidence, three correct, gives ECE = 0.15).

7.2 Latency measurement protocol

- Model in eval mode, torch.inference_mode(), float32. The same 1,024 real test images are pre-loaded onto the GPU, so data loading and CPU-to-GPU copies are excluded from the timings.
- 20 untimed warm-up batches per mode (to finish GPU start-up, memory allocation and kernel loading).
- Each batch is timed separately, with torch.cuda.synchronize() just before and just after it, because GPU calls return before the GPU has finished; without synchronising, timings would be meaningless.
- All 1,024 images are processed 5 times; we report the median and mean ± standard deviation of batch latency, the 90th percentile, throughput, and the spread of throughput across repeats.
- **Batch size 1** measures true per-image latency (interactive use). **Batch size 128** measures throughput; its per-image figure is batch time ÷ 128 (amortised), not the waiting time of one image.
- cuDNN autotuning is disabled during timing, so the adaptive model's changing batch sizes are not penalised by one-off tuning, and every mode runs under identical settings on the same hardware.

7.3 Multiple seeds

Every CIFAR-10 experiment was repeated with three training seeds. Tables report mean ± sample standard deviation. Three seeds are enough to see whether a difference is larger than run-to-run variation, but not to make formal claims, so **no statistical significance test was performed** and none is claimed.

8. Results

8.1 Main comparison on CIFAR-10

Method	Test accuracy (%)	MACs (% of full)	Mean exit depth	ECE	Early-exit error (%)
Standard CNN (Model A)	90.37 ± 0.10	100.0 ± 0.0	3.00 ± 0.00	0.045 ± 0.002	–
Multi-exit, Exit 3 only	90.11 ± 0.20	100.0 ± 0.0	3.00 ± 0.00	0.041 ± 0.002	–
Multi-exit, Exit 2 only	85.89 ± 0.33	62.9 ± 0.0	2.00 ± 0.00	0.019 ± 0.004	–
Multi-exit, Exit 1 only	67.44 ± 0.14	25.9 ± 0.0	1.00 ± 0.00	0.019 ± 0.001	–
Adaptive, $\tau = 0.9$	89.72 ± 0.30	68.6 ± 0.2	2.15 ± 0.01	0.033 ± 0.003	3.25 ± 0.19
Adaptive, val-selected τ (≈ 0.85)	89.28 ± 0.24	63.5 ± 0.8	2.01 ± 0.02	0.027 ± 0.003	4.93 ± 0.26

Table 6. CIFAR-10 test results, mean ± std over 3 seeds.

Reading the table. The baseline and the multi-exit model's final exit are close (90.37% vs 90.11%); the 0.26-point gap is about the size of the seed-to-seed spread, so the extra heads cost little or nothing in final accuracy. Fixed shallow exits are much cheaper but much less accurate (Exit 1: 67.4% for 26% of the computation). The adaptive policy recovers almost all of the full network's accuracy (89.3%) for 63.5% of the computation.

Key result
At almost exactly the same estimated computation (≈63%), always using Exit 2 gives 85.89 ± 0.33% while adaptive inference gives 89.28 ± 0.24%, about 3.4 points better. Deciding the depth per image is far more efficient than choosing a single shallower depth for all images.

8.2 Results per seed

Seed	Standard CNN	Exit 1	Exit 2	Exit 3	Adaptive τ=0.9	Adaptive selected (τ)	Selected MACs
0	90.34	67.59	85.70	90.04	89.59	89.08 (0.84)	62.6%
1	90.49	67.42	85.70	89.95	89.50	89.20 (0.86)	64.2%
2	90.29	67.32	86.28	90.33	90.06	89.55 (0.85)	63.7%

Table 7. Test accuracy (%) for each training seed.

The ranking of methods is the same for every seed, and the spread between seeds is small (≤0.33 points), so the conclusions do not depend on one lucky run.

8.3 Effect of the threshold (RQ2, RQ3)

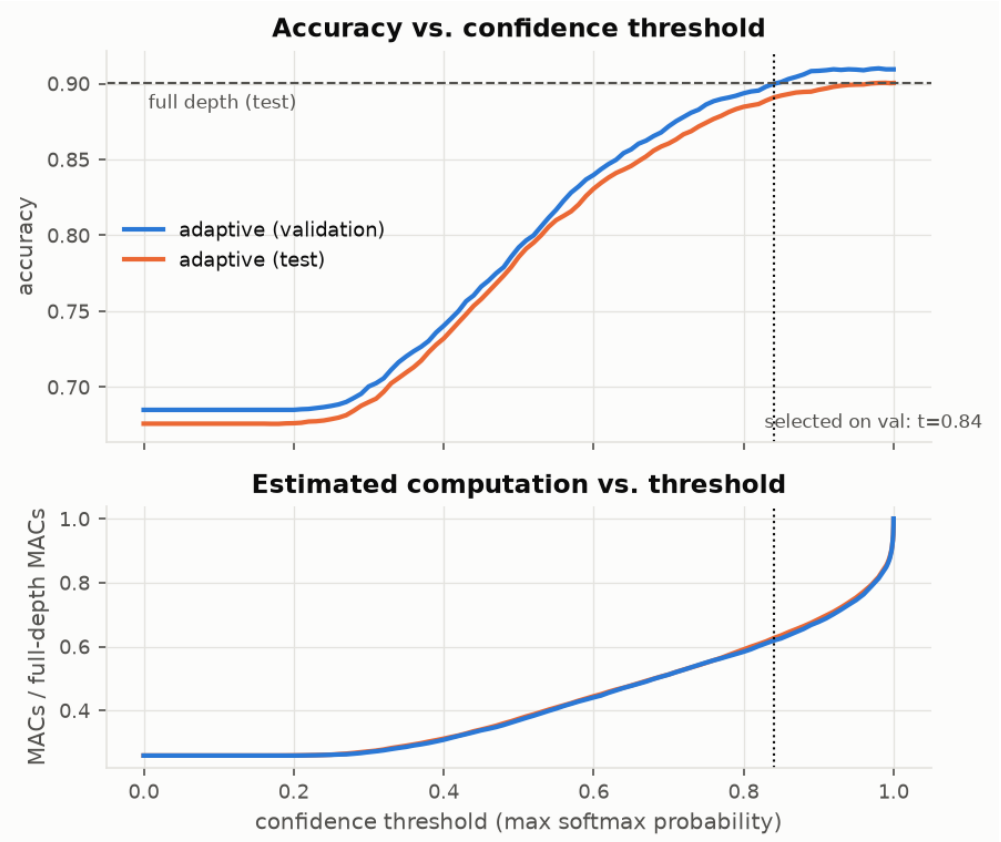


Figure 3. Accuracy (top) and estimated computation (bottom) versus threshold, seed 0. Dotted line: threshold selected on validation data.

τ	Test acc (%)	MACs (% full)	Exit 1 (%)	Exit 2 (%)	Exit 3 (%)	Early-exit error (%)
0.0	67.59	25.9	100.0	0.0	0.0	32.41
0.5	78.57	37.4	72.5	23.9	3.5	20.79
0.7	86.05	51.3	45.9	39.7	14.4	10.85
0.8	88.47	59.1	32.6	45.2	22.2	6.71
0.84	89.08	62.6	26.8	47.2	26.0	5.23
0.9	89.59	68.6	17.8	49.3	32.9	3.17
0.95	89.93	75.3	8.9	48.8	42.3	1.63
0.99	90.05	85.3	0.8	38.1	61.1	0.51
0.999	90.04	93.3	0.0	18.3	81.7	0.05
1.0	90.04	100.1	0.0	0.0	100.0	–

Table 8. Threshold sweep on the test set, seed 0 (descriptive only; the threshold was chosen on validation).

As τ increases, fewer images stop early, computation rises and accuracy rises, then saturates. Above τ ≈ 0.99 accuracy no longer improves, but computation keeps climbing to 100%. The useful region is roughly τ = 0.8–0.95. At τ = 0 every image stops at Exit 1 (67.6%); at τ = 1.0 every image reaches Exit 3 (90.0%).



Figure 4. Fraction of test images leaving at each exit as the threshold changes (seed 0).

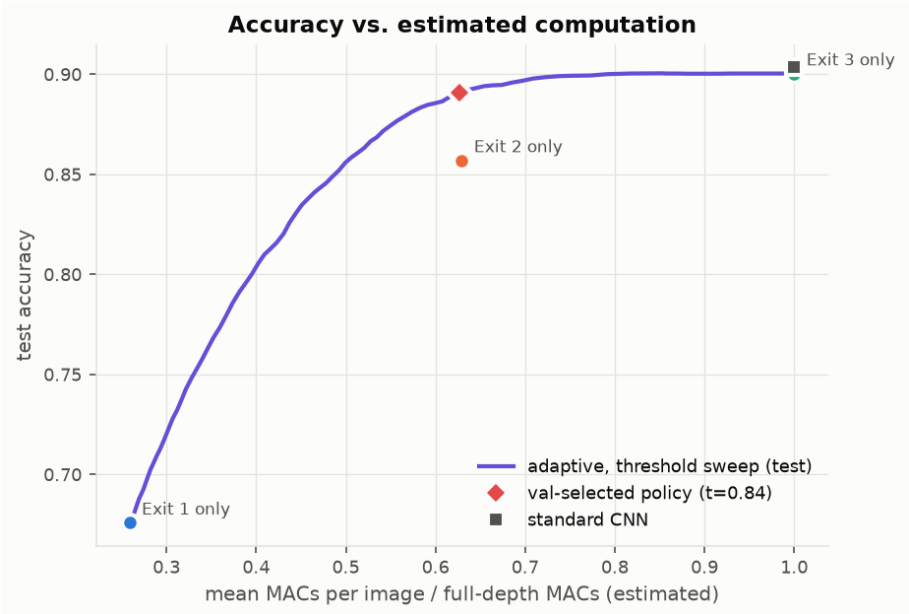


Figure 5. Accuracy versus estimated computation. The adaptive curve lies well above the straight path between fixed exits, which is the benefit of choosing depth per image.

8.4 Which images stop early?

Exit	Share of test images	Accuracy on those images
Exit 1	26.8%	94.0%
Exit 2	47.2%	95.2%
Exit 3	26.0%	72.9%
All	100%	89.1%

Table 9. Where images exit and how accurate each group is (seed 0, τ = 0.84).

This table shows the mechanism working as intended. The images that stop at Exit 1 are classified correctly 94% of the time, much better than Exit 1's overall 67.6%, because the confidence filter lets through only the easy images. The images that travel to Exit 3 are the hard ones: even the full network gets only 72.9% of them right, compared with 90.0% on the whole test set. In other words, confidence is a useful (though imperfect) measure of difficulty.

8.5 Per-class accuracy

Class	airplane	automobile	bird	cat	deer	dog	frog	horse	ship	truck
Exit 1	71	81	47	40	64	64	79	70	82	77
Exit 2	88	94	79	70	85	78	92	87	93	91
Exit 3	93	96	85	76	90	86	94	92	95	93

Table 10. Per-class test accuracy (%) of each exit, seed 0.

Vehicles (automobile, ship, truck) and frogs are easiest at every depth, because colour and shape are distinctive. Cat, bird and dog are hardest; they gain the most from depth (cat: 41% → 70% → 77%). The confusion matrix (Appendix C) shows that the most common confusion is cats predicted as dogs (110 of 1,000 cat images for seed 0), followed by the reverse.

8.6 Calibration (RQ6)

Exit / policy	Accuracy (%)	Mean confidence (%)	ECE	NLL
Exit 1	67.59	66.17	0.0186	0.908
Exit 2	85.70	87.76	0.0214	0.411
Exit 3	90.04	94.14	0.0411	0.327
Adaptive (τ = 0.84)	89.08	–	0.0273	–

Table 11. Calibration of each exit on the test set (seed 0).

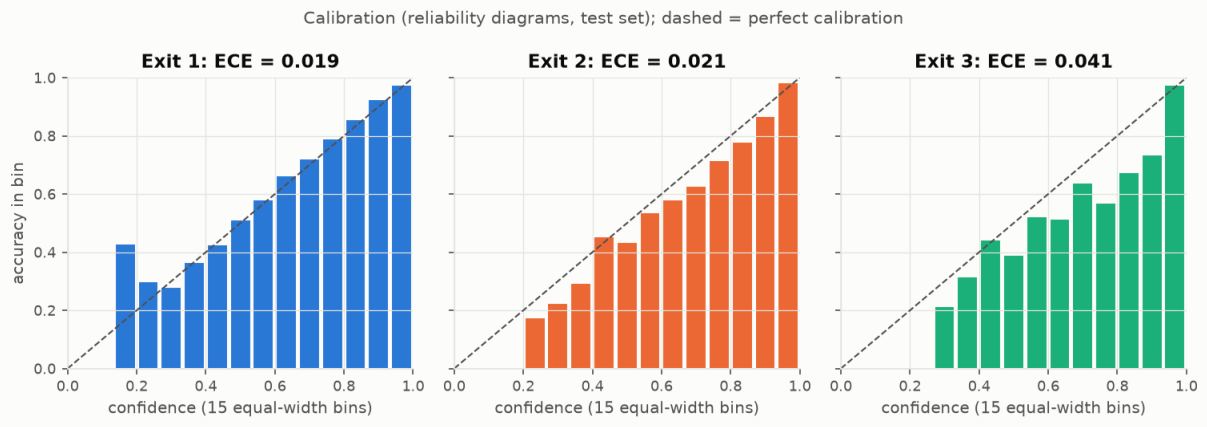


Figure 6. Reliability diagrams. Bars below the dashed diagonal mean the model is overconfident in that bin.

Exit 3 is the most accurate but also the most overconfident (mean confidence 94.1% vs accuracy 90.0%, ECE 0.041), which matches Guo et al.'s observation that deeper, more accurate networks tend to be overconfident. The shallow exits are better calibrated (ECE ≈ 0.02). The adaptive policy inherits part of this: its ECE (0.027 ± 0.003 averaged over seeds) is lower than the final exit's (0.041 ± 0.002). The standard CNN has the highest ECE (0.045 ± 0.002).

8.7 Latency on the GPU (RQ4)

Method	bs=1: ms per image	bs=1: images/s	bs=128: ms per image (amortised)	bs=128: images/s
Standard CNN (Model A)	0.650 ± 0.017	1228 ± 138	0.0398 ± 0.0018	24922 ± 512
Multi-exit, Exit 3 only	0.611 ± 0.011	1432 ± 60	0.0359 ± 0.0002	27725 ± 197
Multi-exit, Exit 2 only	0.433 ± 0.030	1896 ± 219	0.0286 ± 0.0001	34717 ± 102
Multi-exit, Exit 1 only	0.275 ± 0.032	2912 ± 548	0.0192 ± 0.0001	51770 ± 350
Adaptive, τ = 0.9	1.058 ± 0.080	863 ± 70	0.0443 ± 0.0028	21583 ± 1225
Adaptive, val-selected τ (≈0.85)	0.963 ± 0.028	947 ± 29	0.0451 ± 0.0043	21714 ± 2178

Table 12. Measured GPU latency (median) and throughput, mean ± std over 3 seeds, RTX 5060 Laptop GPU.

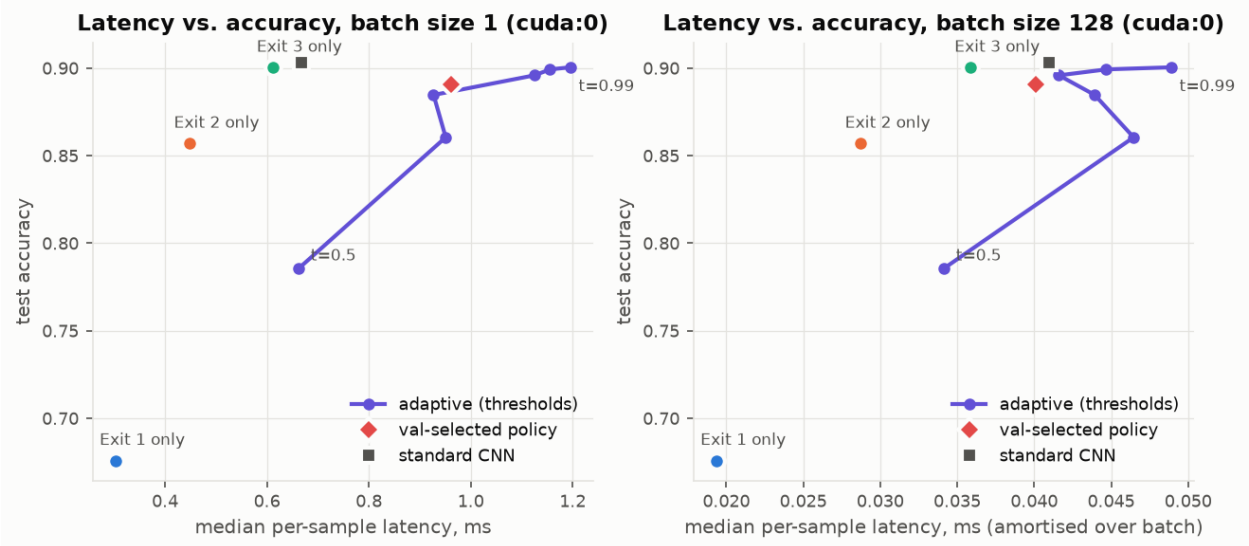


Figure 7. Accuracy versus measured latency (seed 0). Unlike Figure 5, the adaptive curve is to the right of the full network: it is slower.

Key result: fewer MACs did not mean less time
Although the adaptive policy needs about 36% fewer MACs, it was **slower** than running the whole network: 0.96 ± 0.03 ms vs 0.61 ± 0.01 ms per image at batch size 1 (≈58% slower), and 21714 ± 2178 vs 27725 ± 197 images/s at batch size 128 (≈22% lower throughput). Fixed exits, which have no decision overhead, do get faster in proportion to their depth.

Why? This network is small, and a modern GPU finishes one small convolution very quickly, so the time is dominated by fixed costs per operation rather than by arithmetic. The adaptive policy adds work that fixed-depth inference does not have: two extra heads and softmax computations, a comparison, a GPU-to-CPU synchronisation at each exit (the CPU must wait to learn how many images finished), and boolean-mask indexing that copies the remaining samples into a new, smaller tensor. At batch size 1 each synchronisation stalls the GPU pipeline; at batch size 128 the later stages run on smaller, less efficient batches. The savings (skipping 26–37% of arithmetic) are smaller than these overheads. On MNIST, where ≈84% of images stop at Exit 1, the adaptive model was slightly faster than full depth at batch size 1 (0.56 vs 0.62 ms), which shows the outcome depends on how many images exit early and on the hardware. On a CPU or a small edge device, where time is closer to proportional to arithmetic, the MAC savings would be more likely to appear as real speed-ups; that was not measured here.

8.8 Robustness to corruption (RQ5)

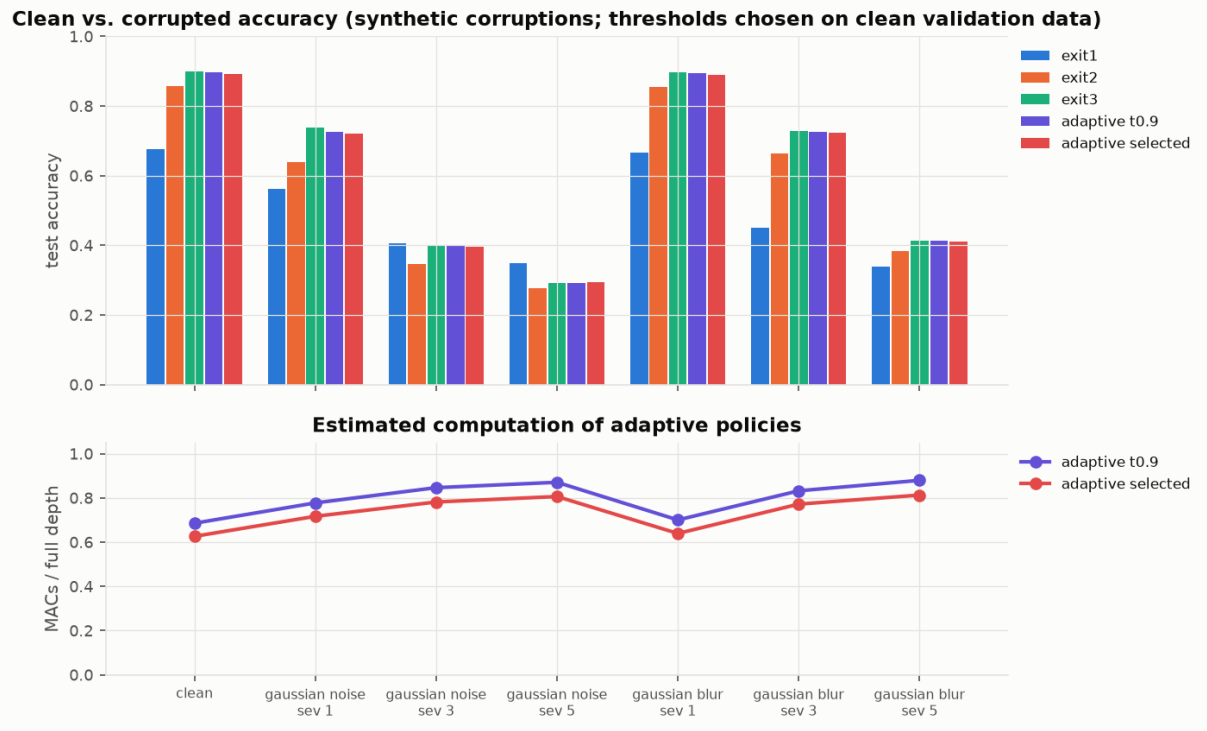


Figure 8. Clean versus corrupted accuracy (top) and computation of the adaptive policies (bottom), seed 0. Thresholds were chosen on clean validation data.

Condition	Standard CNN	Exit 1	Exit 3	Adaptive (selected)	Adaptive MACs (% full)	Early-exit error (%)
Clean	90.4	67.4	90.1	89.3	63.5	4.9
Gauss. noise sev 1	72.2	57.0	73.2	72.0	72.8	16.8
Gauss. noise sev 3	37.9	40.3	37.6	37.8	79.2	47.2
Gauss. noise sev 5	27.3	35.0	25.6	27.2	81.4	53.9
Gauss. blur sev 1	90.2	66.5	89.8	89.1	65.0	4.7
Gauss. blur sev 3	70.9	45.9	70.9	70.4	79.1	13.6
Gauss. blur sev 5	40.4	35.1	38.9	38.7	82.1	49.0

Table 13. Accuracy (%) under synthetic corruption, mean over 3 seeds.

Three observations. (1) The network itself is fragile to noise: even the mildest noise ($\sigma = 0.04$) drops full-depth accuracy from 90% to about 73%, and the standard CNN suffers the same way. This is typical of CNNs trained without noise augmentation. (2) The adaptive policy stays close to the full network under every corruption (within about 1 point, sometimes slightly above), and it **automatically spends more computation on corrupted images**: average MACs rise from 63% (clean) to about 81% (strongest noise or blur), because confidence falls and fewer images pass the early thresholds. (3) Under strong noise, Exit 1 is actually more accurate than Exit 3, so deeper is not always better on shifted data. However, the early-exit error rate rises sharply (from about 5% on clean images to roughly 47–54% at noise severity 3–5 and 49% at blur severity 5): when an early exit is confident on a corrupted image, it is often wrong. Confidence is therefore a weaker guide to correctness under distribution shift.

On MNIST the picture was worse for the adaptive model: at the strongest noise, adaptive accuracy fell to about 42–55% while Exit 3 alone kept 85%, because the MNIST Exit 1 stayed confident on noisy digits it got wrong. The reliability of confidence-based exiting therefore depends on the dataset and the type of shift.

8.9 MNIST smoke test

Method (2 epochs, 1 seed)	Test accuracy (%)	MACs (% full)
Standard CNN	98.82	100
Multi-exit, Exit 1 / 2 / 3 only	96.31 / 98.85 / 99.09	25 / 63 / 100
Adaptive $\tau = 0.9$	98.91	32.3
Adaptive, val-selected $\tau = 0.77$ (≤0.5-pt drop)	98.51	28.7

Table 14. MNIST smoke-test results (short run used to validate the pipeline).

MNIST is so easy that 84% of digits stop at Exit 1 at $\tau = 0.9$, so computation drops to about one third with only a 0.2-point accuracy loss. This shows that the benefit of early exiting grows when more of the data is easy.

9. Answers to the research questions

RQ1. Conventional CNN vs CNN with intermediate classifiers?

Final accuracy is practically the same (90.37 ± 0.10% vs 90.11 ± 0.20%); the 0.26-point difference is within run-to-run variation. The multi-exit model adds 7.9% parameters but only 0.07% MACs, and gives two extra cheaper predictions for free. Its final exit is also slightly better calibrated (ECE 0.041 vs 0.045).

RQ2. How does the threshold affect accuracy and cost?

Monotonically: a higher threshold means fewer early exits, more computation and higher accuracy, with diminishing returns above about 0.95. The threshold is a single control knob for the accuracy–efficiency trade-off.

RQ3. Can early exits reduce average computation?

Yes. With the validation-selected threshold, average estimated MACs fell to 63.5 ± 0.8% of the full network at a cost of about 0.8 accuracy points; at $\tau = 0.9$ the cost was 68.6 ± 0.2% with about 0.4 points lost.

RQ4. Does it reduce measured GPU latency?

No, not in this setting. On the RTX 5060 adaptive inference was slower than full-depth inference at both batch sizes, because decision and batch-splitting overheads outweighed the skipped arithmetic for this small network. Fixed shallow exits were faster. Only on MNIST, where most images exit at Exit 1, was a small batch-1 speed-up seen.

RQ5. Are early exits reliable under corruption?

Partly. On CIFAR-10 the policy reacted sensibly, sending more images deeper and keeping accuracy close to full depth, but confident early exits on corrupted images were wrong much more often (early-exit error rose from about 5% to roughly 50%). On MNIST under strong noise the policy was clearly worse than full depth. Maximum softmax probability is not a reliable correctness signal under distribution shift.

RQ6. How do the metrics change across operating conditions?

Lower thresholds trade accuracy for computation; corruption increases computation automatically and lowers accuracy for every method; batch size changes the latency picture but not the ranking (adaptive slower than full depth); shallow exits are better calibrated than the final exit.

9.1 Overall conclusion

Early exiting works as an *algorithm*: the confidence signal separates easy from hard images well enough to save about a third of the arithmetic while keeping accuracy within about one point, and it beats a fixed shallow network at equal cost. But arithmetic savings are not automatically time savings. On a GPU with a small CNN, the overhead of dynamic decisions dominated. Whether early exiting is worthwhile in practice depends on the hardware, the batch size, the fraction of easy inputs and how expensive the skipped layers are, and it must be measured rather than assumed.

10. Limitations and future work

10.1 Limitations

- Small network and small images: savings and overheads may differ greatly for large models such as ResNet-50 or Vision Transformers, where each stage is far more expensive than the decision overhead.
- Latency was measured on a single laptop GPU running other applications; absolute numbers will change on other hardware, and the laptop's power and heat management add noise (reflected in the standard deviations).
- Only three seeds and no significance testing.
- One shared threshold and one confidence score (max softmax probability). No per-exit thresholds were tuned and no learned exit policy was tried.
- Synthetic corruptions only; the real CIFAR-10-C benchmark is supported in the code but was not evaluated.
- 30 epochs with a single default hyperparameter setting; no hyperparameter search, so both models could likely be made more accurate.
- MAC estimates count only convolutions and linear layers; batch norm, ReLU, pooling and softmax are ignored (they are small but not zero).

10.2 Future work

- Measure on CPU and on an edge device (for example a Raspberry Pi or Jetson), where latency tracks arithmetic more closely.
- Reduce GPU overhead: CUDA graphs, TensorRT, or sorting a stream of requests by exit to keep batches full.
- Alternative confidence scores: entropy, the gap between the top two probabilities, or temperature-scaled probabilities to fix overconfidence.
- Different thresholds per exit, or a small learned network that decides whether to exit.
- Better training of early exits: knowledge distillation from the final exit, or weighting schemes that change during training.
- Train with noise augmentation and evaluate on the full CIFAR-10-C to improve and measure robustness.

11. Implementation

11.1 Repository structure

```
ThinkOrStop/
  run_all.ps1            one-command CIFAR-10 pipeline
  pyproject.toml, requirements.txt, .gitignore
  data/raw/              MNIST and CIFAR-10 (downloaded by torchvision)
  checkpoints/<run>/     best.pt and last.pt for each run
  results/logs/<run>/    train.log, history.csv/json, config.json
  results/metrics/<run>/ evaluation outputs (CSV/JSON), report_cifar10.md
  results/figures/<run>/ PNG figures
  src/think_or_stop/
    config.py       all settings and command-line flags
    datasets.py     loading, train/val split, corruptions, CIFAR-10-C
    models.py       CNN backbone, exit heads, MAC counting
    losses.py       weighted multi-exit loss
    train.py        training loop, checkpoints, resume
    inference.py    fixed-exit and adaptive (early-exit) inference
    thresholds.py   validation-only threshold selection
    metrics.py      accuracy, confusion matrix, ECE, adaptive summary
    benchmark.py    latency measurement
    evaluate.py     complete evaluation command
    plots.py        figures
    report.py       multi-seed summary tables
    predict.py      classify your own images
    gpu_check.py    GPU verification and smoke test
    download.py     explicit dataset download
  tests/            41 automated tests (pytest)
```

11.2 Testing

The project has 41 automated tests, all passing on this machine (including the CUDA tests). They cover: output shapes for MNIST and CIFAR-10 and for both models; that fixed-exit inference equals the full forward pass; MAC counts against hand calculations; the weighted loss and gradients reaching every head; a CPU training smoke test; single-image and batched early exit; correct ordering after regrouping and shuffling; proof that exited samples skip deeper stages; the incomplete last batch; determinism and absence of gradients; per-exit thresholds; threshold selection (feasible, infeasible and validation-only); ECE and the confusion matrix on hand-made examples; reproducible data splits and deterministic corruptions; checkpoint save/load; configuration parsing; train-then-resume end to end; and GPU forward/backward and GPU–CPU agreement.

11.3 Main commands

```
# whole pipeline (3 seeds, 30 epochs)
powershell -ExecutionPolicy Bypass -File .un_all.ps1

# individual steps (PowerShell; ` continues a line)
python -m think_or_stop.gpu_check --device cuda
python -m think_or_stop.train --dataset cifar10 --model baseline `
       --device cuda --epochs 30 --batch-size 128
python -m think_or_stop.train --dataset cifar10 --model multiexit `
       --device cuda --epochs 30 --batch-size 128
python -m think_or_stop.evaluate `
       --checkpoint checkpoints/cifar10_multiexit_s0/best.pt --device cuda `
       --threshold 0.9 --select-threshold --constraint relative `
       --max-drop 0.01 --corruptions synthetic
python -m think_or_stop.plots --metrics-dir results/metrics/cifar10_multiexit_s0 `
       --baseline-dir results/metrics/cifar10_baseline_s0
python -m think_or_stop.report --out results/metrics/report_cifar10 `
       --metrics-dirs (Get-ChildItem results/metrics -Directory -Filter "cifar10_*").FullName
python -m think_or_stop.predict my_photo.jpg      # classify your own image
python -m pytest -q                               # run the tests
```

12. Viva and cross-question preparation

Likely examiner questions with short, accurate answers. Numbers refer to the CIFAR-10 results above.

12.1 Concept questions

Q1. What is an early-exit network in one sentence?

A network with classifiers at several depths that lets each input stop at the first classifier that is confident enough, so easy inputs use less computation than hard ones.

Q2. Is this your own invention?

No. Early exiting is established: BranchyNet (2016), MSDNet (2018) and Shallow-Deep Networks (2019) are the key papers. My work is a from-scratch implementation and a careful evaluation, including GPU latency, calibration and corruption, where I report the negative results honestly.

Q3. Why use the maximum softmax probability as confidence?

It is the simplest score, costs almost nothing, needs no extra training and is standard in the literature. Its weakness is that networks can be overconfident, so it is only a proxy for correctness. Alternatives are entropy (used by BranchyNet), the margin between the top two probabilities, temperature scaling, or a learned exit decision.

Q4. Why train all exits together rather than training the backbone first and then adding heads?

Joint training lets the early stages learn features that are useful for an immediate decision, not only as input for later layers, so the early exits become much better. The cost is that the shared layers must satisfy several objectives; here that cost was small (final accuracy 90.11% vs 90.37% for the baseline).

Q5. Why are the loss weights 1, 1, 1?

Equal weights are the most neutral choice and need no tuning. They are configurable. Larger weights on the final exit would protect final accuracy; larger early weights would improve early exits. Some papers change the weights during training. I did not tune them, which is listed as a limitation.

Q6. Why does the baseline exist if the multi-exit model has an Exit 3 anyway?

To measure whether adding the extra heads and losses hurts the final classifier. The baseline has exactly the same backbone and final head, trained with plain cross-entropy, so it is the fair 'conventional CNN' reference.

Q7. What does each layer type do, and why batch norm and dropout?

Convolutions learn local patterns with shared weights; ReLU adds non-linearity; max pooling halves resolution and adds shift tolerance; fully connected layers combine features into class scores. Batch norm stabilises and speeds up training; dropout (0.3) in the heads reduces overfitting of the fully connected layers.

Q8. Why adaptive average pooling in the heads?

It reduces any feature-map size to 2×2, so the same head design works for MNIST and CIFAR-10 and the number of head parameters stays small and independent of image size.

Q9. Adam or SGD?

Both are implemented. Adam with learning rate 0.001 was used for all results because it converges reliably without tuning. Well-tuned SGD with momentum often generalises slightly better on CIFAR-10; comparing them is possible with the --optimizer flag.

Q10. Why cosine learning-rate decay?

It reduces the learning rate smoothly to zero over a fixed number of epochs, which usually improves final accuracy compared with a constant rate and requires no extra settings such as step points.

12.2 Methodology questions

Q11. How did you avoid test-set leakage?

A 5,000-image validation set is held out from the training data with a fixed split seed. Checkpoint selection and threshold selection use only validation data. The selected threshold is applied unchanged to the test set. The threshold-selection function does not even accept test data. Test-set sweeps are reported only as descriptive curves.

Q12. How exactly is the threshold selected?

I try thresholds from 0 to 1 on validation data, keep those whose accuracy is within 1 point of the full network's validation accuracy, and choose the one with the lowest average MACs. If none qualifies, the fallback is to never exit early. The three seeds chose 0.84, 0.86 and 0.85.

Q13. What does 'relative' constraint mean versus 'absolute'?

Relative: adaptive accuracy must be at least the same model's full-depth validation accuracy minus a tolerance (1 point here). Absolute: adaptive accuracy must reach a fixed number, such as 88%. Relative can always be met (by never exiting early); absolute might be impossible, which the code reports.

Q14. How do you compute MACs, and why MACs and not FLOPs?

Forward hooks record each convolution's and linear layer's output size; convolution MACs = output channels × height × width × input channels × 3 × 3; linear MACs = inputs × outputs. FLOPs are about 2 × MACs, so they carry the same information. For adaptive inference I also count every head an image passed through.

Q15. How did you measure latency, and why synchronise?

The model is in eval and inference mode, images are pre-loaded on the GPU, there are 20 warm-up batches, and each batch is timed with torch.cuda.synchronize() before and after, because GPU calls are asynchronous and return before the work is done. I repeat 5 times and report the median, mean and standard deviation, at batch sizes 1 and 128, on the same hardware for every method.

Q16. What is the difference between per-sample latency and amortised latency?

At batch size 1, latency is the real time one image waits. At batch size 128, I divide the batch time by 128; that is a throughput measure, not the waiting time of one image (which is the whole batch time).

Q17. How do you know the batched early-exit code is correct?

Tests compare it with a one-image-at-a-time reference, shuffle the batch to check ordering, use hooks to prove exited images never enter deeper stages, and check the final incomplete batch. On the real test set it gave 100% identical predictions to a simulation from stored outputs.

Q18. Why does a sample's result not depend on the other samples in its batch?

In evaluation mode batch norm uses stored running statistics instead of the current batch's statistics, and dropout is off, so each image is processed independently.

Q19. Why three seeds, and is the difference significant?

Three seeds show whether results are stable across random initialisations; the method ranking was the same in all three. I report mean ± standard deviation but did not run a significance test, so I do not claim statistical significance; differences smaller than the spread (such as 90.37 vs 90.11) should be treated as 'no clear difference'.

12.3 Results questions

Q20. What is your single most important result?

At the same estimated computation (about 63%), adaptive inference reached 89.3% while a fixed Exit 2 reached 85.9%. But on the GPU the adaptive model was slower than the full network, so the computation savings did not become time savings.

Q21. Why is adaptive inference slower on the GPU if it does less work?

The network is small, so each GPU operation is fast and the time is dominated by fixed costs: extra head evaluations and softmax, a GPU-to-CPU synchronisation at each exit, and copying the remaining samples into smaller tensors. Skipping 37% of the arithmetic saves less time than these overheads add. FLOPs are not latency.

Q22. When would early exiting give a real speed-up?

When the skipped layers are expensive compared with the decision overhead (large models, high-resolution inputs), when most inputs are easy (as on MNIST, where a small batch-1 speed-up appeared), on CPUs or edge devices where time follows arithmetic more closely, or with engineering such as CUDA graphs or grouping requests by exit.

Q23. Why is the final exit's accuracy only 72.9% on the images that reach it?

Because the policy sends only the hard images there; the easy ones already left. Those hard images are hard for every exit. It shows the confidence score is separating easy from hard images.

Q24. Why is Exit 1 so weak (67%)?

It sees only two convolution layers at full resolution, with a small receptive field and low-level features (edges, colours). That is enough for distinctive classes like automobile and ship (about 81%) but not for cat (41%) or bird (47%).

Q25. What do ECE values tell us?

Exit 3 is overconfident (94% average confidence vs 90% accuracy, ECE 0.041). Early exits are better calibrated (about 0.02). The adaptive policy's ECE (0.027) is better than the final exit's, so its confidence is more trustworthy on clean data.

Q26. Why do early exits fail under noise?

The network never saw noise during training. Corruption lowers confidence on average, so more images go deeper (computation rose from 63% to about 81%), but the images that still pass the threshold are often confidently wrong: the early-exit error rose from about 5% to roughly 50% at the strongest corruptions. Softmax confidence is unreliable under distribution shift.

Q27. Is the threshold chosen on clean data still valid for noisy data?

Not necessarily; that is exactly what the experiment tests. I deliberately did not re-tune on corrupted data, because in deployment you do not know the corruption in advance and using corrupted test labels would be leakage. The results show accuracy stayed close to full depth on CIFAR-10 but not on MNIST.

Q28. Your model is overfitting? Training 96% vs validation 91%.

There is a moderate generalisation gap, normal for CIFAR-10 without heavy regularisation. Validation accuracy kept rising until the last epoch, so it is not overfitting in the harmful sense. More augmentation or weight decay could reduce the gap.

Q29. What is the accuracy of 90% compared with the state of the art?

State-of-the-art CIFAR-10 models reach over 99%, usually with large networks, long training and pretraining. This project's goal was a compact model trained from scratch in minutes to study early exiting, not maximum accuracy. 90% is reasonable for a 1.3-million-parameter VGG-style network after 30 epochs.

12.4 Implementation questions

Q30. How long did training take and how much GPU memory?

About 30 epochs × 6–12 s per model, a few minutes per run and about 35 minutes for all 6 runs plus evaluation. Peak GPU memory was under 1 GB of the available 8 GB.

Q31. How did you make sure the GPU was really used?

The code refuses to run if CUDA is requested but unavailable, prints the GPU name, and logs the device of the model parameters and the first batch (cuda:0). A GPU check script also runs a one-batch forward and backward pass on the GPU.

Q32. Can the experiment be reproduced?

Yes. Seeds are fixed, the validation split has its own seed, every setting is saved in config.json and inside each checkpoint, and run_all.ps1 reruns the whole pipeline. GPU results are statistically, not bit-for-bit, identical unless --deterministic is used.

Q33. What happens with the last batch that is smaller than 128?

Nothing special is needed: the code works on any batch size, and a test checks that 23 images split into batches of 8, 8 and 7 give the same results as one batch of 23.

Q34. Why no pretrained model?

The project requirement was to train from scratch. Pretrained features would also hide how much each depth can learn on its own, which is the point of studying exits.

Q35. If you had more time, what would you change first?

Measure latency on a CPU and an edge device, try entropy and temperature-scaled confidence, tune per-exit thresholds, and evaluate on the real CIFAR-10-C.

References

- S. Teerapittayanon, B. McDanel and H. T. Kung. BranchyNet: Fast inference via early exiting from deep neural networks. International Conference on Pattern Recognition (ICPR), 2016.
- G. Huang, D. Chen, T. Li, F. Wu, L. van der Maaten and K. Q. Weinberger. Multi-scale dense networks for resource efficient image classification. International Conference on Learning Representations (ICLR), 2018.
- Y. Kaya, S. Hong and T. Dumitras. Shallow-Deep Networks: Understanding and mitigating network overthinking. International Conference on Machine Learning (ICML), 2019.
- C. Guo, G. Pleiss, Y. Sun and K. Q. Weinberger. On calibration of modern neural networks. ICML, 2017.
- D. Hendrycks and T. Dietterich. Benchmarking neural network robustness to common corruptions and perturbations. ICLR, 2019. Dataset: doi:10.5281/zenodo.2535967.
- A. Krizhevsky. Learning multiple layers of features from tiny images. Technical report, University of Toronto, 2009 (CIFAR-10).
- Y. LeCun, L. Bottou, Y. Bengio and P. Haffner. Gradient-based learning applied to document recognition. Proceedings of the IEEE, 1998 (MNIST).
- S. Ioffe and C. Szegedy. Batch normalization: Accelerating deep network training by reducing internal covariate shift. ICML, 2015.
- N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever and R. Salakhutdinov. Dropout: A simple way to prevent neural networks from overfitting. JMLR, 2014.
- D. P. Kingma and J. Ba. Adam: A method for stochastic optimization. ICLR, 2015.
- K. He, X. Zhang, S. Ren and J. Sun. Delving deep into rectifiers (Kaiming initialisation). ICCV, 2015.
- I. Loshchilov and F. Hutter. SGDR: Stochastic gradient descent with warm restarts (cosine schedule). ICLR, 2017.
- K. Simonyan and A. Zisserman. Very deep convolutional networks for large-scale image recognition (VGG). ICLR, 2015.
- A. Paszke et al. PyTorch: An imperative style, high-performance deep learning library. NeurIPS, 2019.

Appendix A. Glossary

Exit / head: a small classifier attached after a stage of the backbone.

Backbone: the shared convolutional layers (Stages 1–3).

Threshold τ: minimum confidence required to stop at an early exit.

Confidence: here, the maximum softmax probability of an exit's prediction.

MAC: multiply-accumulate operation; FLOPs ≈ 2 × MACs.

Mean exit depth: average exit number used (1 to 3).

Early-exit error rate: error rate among the images that stopped at Exit 1 or Exit 2.

ECE: Expected Calibration Error: average gap between confidence and accuracy over confidence bins.

NLL: negative log-likelihood: average −log of the probability given to the true class.

Latency: wall-clock time to process an input; throughput: inputs processed per second.

Validation set: data held out from training, used for choices such as checkpoint and threshold.

Seed: number that fixes random initialisation, shuffling and augmentation for reproducibility.

Distribution shift: test data that differs from training data (for example noise or blur).

Appendix B. Environment

Item	Value
GPU	NVIDIA GeForce RTX 5060 Laptop GPU, 7.93 GB, compute capability 12.0
PyTorch	2.14.1+cu130 (CUDA 13.0, cuDNN 92400)
Python	3.14.7
OS	Windows-11-10.0.26300-SP0
Other libraries	torchvision 0.29.1, NumPy 2.5.2, matplotlib 3.11.2, pytest 9.1.1

Table 15. Software and hardware used for all results.

Appendix C. Confusion matrix

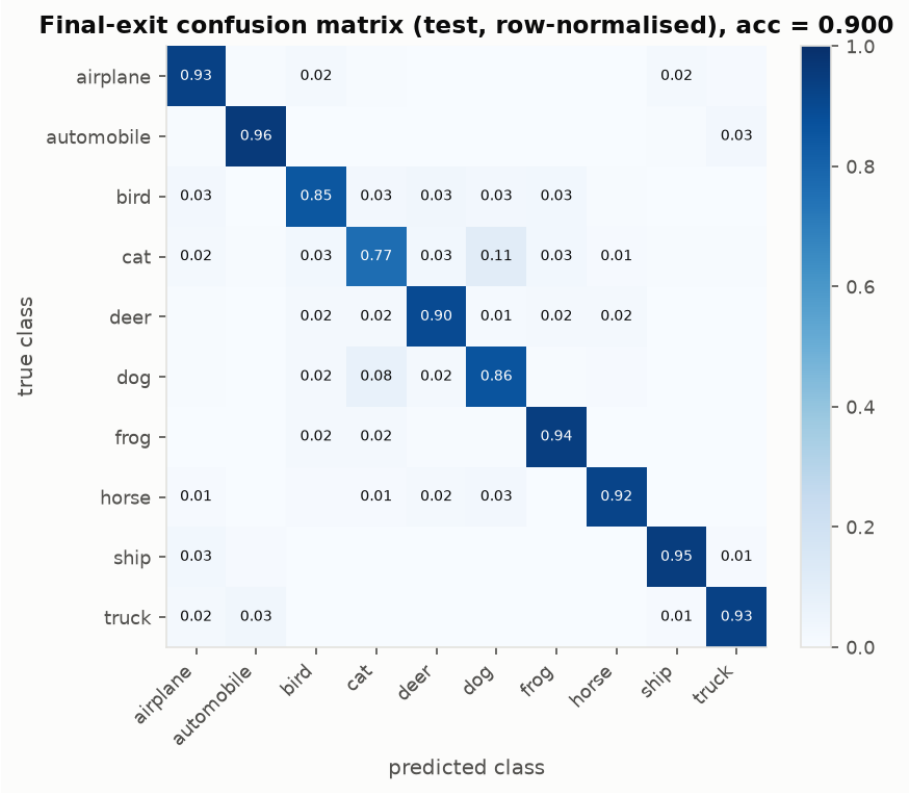


Figure 9. Final-exit confusion matrix on the CIFAR-10 test set (seed 0), row-normalised.